

Ingénierie Dirigée par les Modèles

2SN-BML S7 2025–2026

Guillaume Dupont

`guillaume.dupont@toulouse-inp.fr`

Toulouse INP – ENSEEIHT



Cette présentation est sous licence Creative Common 4.0
BY-NC-SA

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

UE IDM – Organisation

Objectifs :

- ▶ *Aspects théoriques* : techniques de conception haut niveau
- ▶ *Aspects pratiques* : maîtrise d'outils (Eclipse + plug-ins)

Déroulement :

- ▶ Séances : 2 CM + 2 TD + 8 TP + TPPR
⇒ *apport théorique en TP!*
- ▶ Partiel + "mini" projet (binômes) + projet (quadrinômes)
⇒ *projet = livrables + rapport + oral (pour les deux)*

Sommaire

- 1 Introduction
- 2 Principes Fondateurs
- 3 EMOF et Ecore
- 4 Sémantiques des Modèles
- 5 Les Réseaux de Pétri
- 6 Transformations de Modèles
- 7 Application : Mini-Projet
- 8 Conclusion

Rudolf Carnap, *Introduction to Semantics* (d'après *Logique et Connaissance Scientifique*, dir. Jean Piaget ¹) :

*Si dans une investigation on se réfère uniquement à celui qui parle, ou, en termes plus généraux, aux usagers du langage, nous attribuons cette investigation à la **pragmatique** [...].*

*Si nous faisons abstraction des usagers du langage et si nous analysons seulement les expressions et leurs significations, nous nous trouvons dans le domaine de la **sémantique**.*

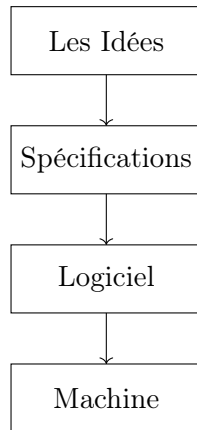
*Et si, finalement, nous faisons abstraction des significations pour analyser uniquement les relations entre expressions, nous entrons dans la **syntaxe**. La totalité de la science du langage, se composant des trois parties mentionnées, forme la **sémio-tique**.*

1. Accessible sur l'Internet Archive :

https://archive.org/details/logiqueetconnais0000unse_16

Motivations

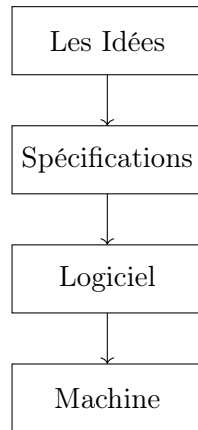
Spécification = *ensemble de règles, de fonctionnalités*



Motivations

Spécification = *ensemble de règles, de fonctionnalités*

Concevoir = *réaliser la **spécification** par un logiciel*



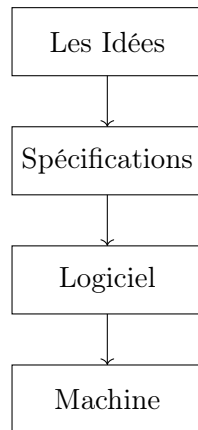
Motivations

Spécification = *ensemble de règles, de fonctionnalités*

Concevoir = *réaliser la **spécification** par un logiciel*

Spécification découle des idées, logiciel découle de spécification...

⇒ *le logiciel a-t-il encore un lien avec les idées ?*



Motivations – Exemple

Spéc. : je désire gérer une bibliothèque

Motivations – Exemple

Spéc. : je désire gérer une bibliothèque

Conception possible :

- ▶ bibliothèque = collection de livres
- ▶ livre = enregistrement (titre, auteur, ISBN, etc.)
- ▶ + UX : champs, tables, impression

Motivations – Exemple

Spéc. : je désire gérer une bibliothèque

Conception possible :

- ▶ bibliothèque = collection de livres
- ▶ livre = enregistrement (titre, auteur, ISBN, etc.)
- ▶ + UX : champs, tables, impression

Concrètement : on a un gros programme avec des enregistrements (ou des classes)

⇒ *les idées ont disparu de la solution*

Motivations – Exemple (suite)

Et maintenant :

Motivations – Exemple (suite)

Et maintenant :

- ▶ Je veux ajouter des fonctionnalités (par ex. gérer des CD)

Motivations – Exemple (suite)

Et maintenant :

- ▶ Je veux ajouter des fonctionnalités (par ex. gérer des CD)
⇒ *reprendre la conception et le développement depuis le début*

Motivations – Exemple (suite)

Et maintenant :

- ▶ Je veux ajouter des fonctionnalités (par ex. gérer des CD)
⇒ *reprendre la conception et le développement depuis le début*
- ▶ Je veux un logiciel compagnon (par ex. imprimer des étiquettes)

Motivations – Exemple (suite)

Et maintenant :

- ▶ Je veux ajouter des fonctionnalités (par ex. gérer des CD)
⇒ *reprendre la conception et le développement depuis le début*
- ▶ Je veux un logiciel compagnon (par ex. imprimer des étiquettes)
⇒ *je vais concevoir des choses redondantes*

Motivations – Exemple (suite)

Et maintenant :

- ▶ Je veux ajouter des fonctionnalités (par ex. gérer des CD)
⇒ *reprendre la conception et le développement depuis le début*
- ▶ Je veux un logiciel compagnon (par ex. imprimer des étiquettes)
⇒ *je vais concevoir des choses redondantes*

Idée : notre logiciel manipule des variables ; mais nous, manipulons des *concepts*

⇒ *peut-on rester au niveau des concepts ? Au niveau du **domaine** ?*

L'Ingénierie Dirigée par les Modèles

Définition

Ensemble de techniques visant à l'automatisation de la conception, la maintenance et/ou l'utilisation de logiciels, en se concentrant sur le *domaine* (modèle) plutôt que sur des implémentations.

L'Ingénierie Dirigée par les Modèles

Définition

Ensemble de techniques visant à l'automatisation de la conception, la maintenance et/ou l'utilisation de logiciels, en se concentrant sur le *domaine* (modèle) plutôt que sur des implémentations.

Principe : *modéliser*, abstraire l'objet du logiciel et son comportement

L'Ingénierie Dirigée par les Modèles

Définition

Ensemble de techniques visant à l'automatisation de la conception, la maintenance et/ou l'utilisation de logiciels, en se concentrant sur le *domaine* (modèle) plutôt que sur des implémentations.

Principe : *modéliser*, abstraire l'objet du logiciel et son comportement

Avantages principaux :

- ▶ haut niveau d'abstraction, automatisation, modularité
- ▶ proximité du *domaine*, de l'utilisateur, *user-in-the-loop*

L'Ingénierie Dirigée par les Modèles

Définition

Ensemble de techniques visant à l'automatisation de la conception, la maintenance et/ou l'utilisation de logiciels, en se concentrant sur le *domaine* (modèle) plutôt que sur des implémentations.

Principe : *modéliser*, abstraire l'objet du logiciel et son comportement

Avantages principaux :

- ▶ haut niveau d'abstraction, automatisation, modularité
- ▶ proximité du *domaine*, de l'utilisateur, *user-in-the-loop*

Inconvénients :

- ▶ potentiellement lourd à mettre en place
- ▶ pas toujours adapté!

L'Ingénierie Dirigée par les Modèles – Utilisation

Les modèles peuvent intervenir à plusieurs points clefs :

- ▶ aide à la conception et/ou au développement (façon UML)
- ▶ structurer les entrées/sorties du logiciel
- ▶ abstraire les interactions utilisateurs
- ▶ abstraire, unifier les traitements sur les données (transformations)

L'Ingénierie Dirigée par les Modèles – Utilisation

Les modèles peuvent intervenir à plusieurs points clefs :

- ▶ aide à la conception et/ou au développement (façon UML)
- ▶ structurer les entrées/sorties du logiciel
- ▶ abstraire les interactions utilisateurs
- ▶ abstraire, unifier les traitements sur les données (transformations)

Possiblement, un logiciel peut être fait uniquement de modèles et de transformations entre ces modèles !

Sommaire

- 1 Introduction
- 2 Principes Fondateurs
 - Les Modèles
 - Syntaxe abstraite, Syntaxes concrètes
 - Méta-modèles, méta-méta-modèle
- 3 EMOF et Ecore
- 4 Sémantiques des Modèles
- 5 Les Réseaux de Pétri
- 6 Transformations de Modèles

Les Modèles

En IDM, tout est un **modèle**.

Les Modèles

En IDM, tout est un **modèle**.

Définition

Un modèle est une représentation **abstraite** d'un objet ou d'un concept, destiné à répondre à des questions ou des besoins.

Les Modèles

En IDM, tout est un **modèle**.

Définition

Un modèle est une représentation **abstraite** d'un objet ou d'un concept, destiné à répondre à des questions ou des besoins.

Remarques :

- ▶ plusieurs modèles pour un même objet
- ▶ modèle = incomplet par nature
- ▶ pas de notion d'implémentation, de stockage, etc.
- ▶ « objet ou concept » = très flou

Un Exemple

Je veux gérer un ensemble de tâches, avec dépendances entre tâches et dépendances à des ressources

$\Rightarrow$ *de quoi ai-je besoin ?*

Un Exemple

Je veux gérer un ensemble de tâches, avec dépendances entre tâches et dépendances à des ressources

⇒ *de quoi ai-je besoin ?*

- ▶ *Des tâches* : développer en JavaScript, bricoler, cuisiner

Un Exemple

Je veux gérer un ensemble de tâches, avec dépendances entre tâches et dépendances à des ressources

⇒ *de quoi ai-je besoin ?*

- ▶ *Des tâches* : développer en JavaScript, bricoler, cuisiner
- ▶ *De la causalité* : « lorsque je commence à développer en JS, je commence à bricoler », « je ne commence à cuisiner que lorsque j'ai terminé de bricoler et de développer en JS »

Un Exemple

Je veux gérer un ensemble de tâches, avec dépendances entre tâches et dépendances à des ressources

⇒ *de quoi ai-je besoin ?*

- ▶ *Des tâches* : développer en JavaScript, bricoler, cuisiner
- ▶ *De la causalité* : « lorsque je commence à développer en JS, je commence à bricoler », « je ne commence à cuisiner que lorsque j'ai terminé de bricoler et de développer en JS »
- ▶ *Des ressources* : deux casseroles, un kit de méditation

Un Exemple

Je veux gérer un ensemble de tâches, avec dépendances entre tâches et dépendances à des ressources

⇒ *de quoi ai-je besoin ?*

- ▶ *Des tâches* : développer en JavaScript, bricoler, cuisiner
- ▶ *De la causalité* : « lorsque je commence à développer en JS, je commence à bricoler », « je ne commence à cuisiner que lorsque j'ai terminé de bricoler et de développer en JS »
- ▶ *Des ressources* : deux casseroles, un kit de méditation
- ▶ *Des besoins* : « pour cuisiner, j'ai besoin d'une casserole », « pour développer en JS, j'ai besoin d'un kit de méditation »

Un Exemple – Leçons

J'ai pris un objet concret (des tâches à réaliser) et je les ai **abstraites**.

Un Exemple – Leçons

J'ai pris un objet concret (des tâches à réaliser) et je les ai **abstraites**.

⇒ *j'ai fait un **modèle** !*

Un Exemple – Leçons

J'ai pris un objet concret (des tâches à réaliser) et je les ai **abstraites**.

⇒ *j'ai fait un **modèle** !*

Des choses sont omises : durée d'une tâche ? lieu ? conditions météo ?

Un Exemple – Leçons

J'ai pris un objet concret (des tâches à réaliser) et je les ai **abstraites**.

⇒ *j'ai fait un **modèle** !*

Des choses sont omises : durée d'une tâche ? lieu ? conditions météo ?

⇒ *je n'ai **que ce dont j'ai besoin** ; c'est ça, l'abstraction !*

Un Exemple – Leçons

J'ai pris un objet concret (des tâches à réaliser) et je les ai **abstraites**.

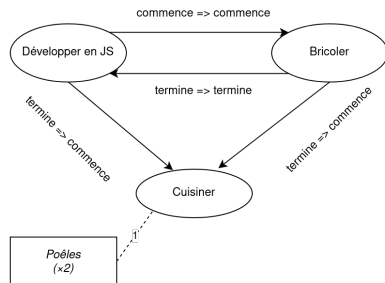
⇒ *j'ai fait un **modèle** !*

Des choses sont omises : durée d'une tâche ? lieu ? conditions météo ?

⇒ *je n'ai **que ce dont j'ai besoin** ; c'est ça, l'abstraction !*

Que faire avec ce modèle ?

- ▶ Le dessiner
- ▶ Extraire une liste de courses
- ▶ Le transformer en emploi du temps
- ▶ *Mangerai-je un jour ?*



Un Peu de Recul

Lorsque j'ai modélisé mes tâches, j'ai fait des catégories, qui correspondent aux **concepts** dont j'ai besoin

⇒ on parle de ***syntaxe abstraite***

Un Peu de Recul

Lorsque j'ai modélisé mes tâches, j'ai fait des catégories, qui correspondent aux **concepts** dont j'ai besoin

⇒ on parle de ***syntaxe abstraite***

Lorsque j'ai fait la liste ou que j'ai dessiné mes tâches, je les ai **représentées** d'une certaine façon

⇒ on parle de ***syntaxes concrètes***

Un Peu de Recul

Lorsque j'ai modélisé mes tâches, j'ai fait des catégories, qui correspondent aux **concepts** dont j'ai besoin

⇒ on parle de ***syntaxe abstraite***

Lorsque j'ai fait la liste ou que j'ai dessiné mes tâches, je les ai **représentées** d'une certaine façon

⇒ on parle de ***syntaxes concrètes***

Remarque (importante) : si je veux modéliser un autre ensemble de tâche, je vais réutiliser ces syntaxes

⇒ *les syntaxes concrètes ne dépendent pas du modèle*

Retour aux Principes Fondateurs

J'ai exprimé mon modèle avec des phrases... Pas très formel!

⇒ ***Question** : comment décrire une syntaxe abstraite ?*

Retour aux Principes Fondateurs

J'ai exprimé mon modèle avec des phrases... Pas très formel!

⇒ **Question** : *comment décrire une syntaxe abstraite ?*

Je veux définir des syntaxes abstraites

⇒ *de quoi ai-je besoin ?*

Retour aux Principes Fondateurs

J'ai exprimé mon modèle avec des phrases... Pas très formel!

⇒ **Question** : *comment décrire une syntaxe abstraite ?*

Je veux définir des syntaxes abstraites

⇒ *de quoi ai-je besoin ?*

- ▶ *Des concepts* (par ex : "procédé", "tâches", "liens de causalité", "ressources", "besoins")

Retour aux Principes Fondateurs

J'ai exprimé mon modèle avec des phrases... Pas très formel!

⇒ **Question** : *comment décrire une syntaxe abstraite ?*

Je veux définir des syntaxes abstraites

⇒ *de quoi ai-je besoin ?*

- ▶ *Des concepts* (par ex : "procédé", "tâches", "liens de causalité", "ressources", "besoins")
- ▶ *Des relations* (par ex : "un procédé se compose de tâches, de ressources et de liens de causalité", "la causalité relie les tâches", "les tâches ont des besoins en ressources")

Retour aux Principes Fondateurs

J'ai exprimé mon modèle avec des phrases... Pas très formel!

⇒ **Question** : *comment décrire une syntaxe abstraite ?*

Je veux définir des syntaxes abstraites

⇒ *de quoi ai-je besoin ?*

- ▶ *Des concepts* (par ex : "procédé", "tâches", "liens de causalité", "ressources", "besoins")
- ▶ *Des relations* (par ex : "un procédé se compose de tâches, de ressources et de liens de causalité", "la causalité relie les tâches", "les tâches ont des besoins en ressources")

Une petite minute! Suis-je en train de définir... Une syntaxe abstraite?

Pour les syntaxes abstraites?!

Prenons de Nouveau du Recul

En IDM,

Prenons de Nouveau du Recul

En IDM, **tout est un modèle.**

Prenons de Nouveau du Recul

En IDM, **tout est un modèle.**

Une syntaxe abstraite est un **modèle** ! Il permet de spécifier d'autres modèles...

⇒ *on l'appelle le **méta-modèle***

Prenons de Nouveau du Recul

En IDM, **tout est un modèle.**

Une syntaxe abstraite est un **modèle** ! Il permet de spécifier d'autres modèles...

⇒ *on l'appelle le **méta-modèle***

On pourrait définir une syntaxe abstraite pour la syntaxe abstraite. Un méta-modèle pour le méta-modèle...

⇒ *on l'appellera le **méta-méta-modèle***

Prenons de Nouveau du Recul

En IDM, **tout est un modèle.**

Une syntaxe abstraite est un **modèle** ! Il permet de spécifier d'autres modèles...

⇒ *on l'appelle le **méta-modèle***

On pourrait définir une syntaxe abstraite pour la syntaxe abstraite. Un méta-modèle pour le méta-modèle...

⇒ *on l'appellera le **méta-méta-modèle***

Remarque : y a-t-il besoin d'aller plus haut ?

⇒ *non ! on peut exprimer le méta-méta-modèle avec lui-même !*

PAUSE.

PAUSE.

On respire.

On Résume

- ▶ Le méta-méta-modèle permet de décrire des méta-modèles (des syntaxes abstraites)
⇒ *en général, fixé, voire standardisé (UML, MOF, etc.)*

On Résume

- ▶ Le méta-méta-modèle permet de décrire des méta-modèles (des syntaxes abstraites)
⇒ en général, fixé, voire standardisé (UML, MOF, etc.)
- ▶ Le méta-modèle permet de décrire des modèles
*⇒ ensemble de concepts et de relations, c'est à nous de les écrire !
c'est **eux** qui répondent aux besoins !*

On Résume

- ▶ Le méta-méta-modèle permet de décrire des méta-modèles (des syntaxes abstraites)
⇒ *en général, fixé, voire standardisé (UML, MOF, etc.)*
- ▶ Le méta-modèle permet de décrire des modèles
⇒ *ensemble de concepts et de relations, c'est à nous de les écrire !
c'est **eux** qui répondent aux besoins !*
- ▶ Les modèles sont des **instances** du méta-modèle (qui respectent ses règles)
⇒ *ce sont les objets que le programme manipule, les entrées, sorties, etc.*

Sommaire

- 1 Introduction
- 2 Principes Fondateurs
- 3 EMOF et Ecore**
- 4 Sémantiques des Modèles
- 5 Les Réseaux de Pétri
- 6 Transformations de Modèles
- 7 Application : Mini-Projet
- 8 Conclusion

Un Standard pour l'IDM ?

Pour faire de l'IDM efficacement, c'est mieux si tout le monde a le même méta-méta-modèle.

Un Standard pour l'IDM ?

Pour faire de l'IDM efficacement, c'est mieux si tout le monde a le même méta-méta-modèle.

⇒ *il existe un standard, par l'OMG : Essential Model-Object Facility (EMOF)*

Un Standard pour l'IDM ?

Pour faire de l'IDM efficacement, c'est mieux si tout le monde a le même méta-méta-modèle.

⇒ *il existe un standard, par l'OMG : Essential Model-Object Facility (EMOF)*

EMOF est un formalisme qui permet de définir des méta-modèles, avec :

- ▶ des **concepts** (*classes*)
- ▶ des **propriétés** (*attributes*)
- ▶ des **références** (*reference*) et ses raffinement (association, composition, ...)
- ▶ de l'**héritage** (*inheritance*)
- ▶ des **types abstraits** (*data-types, enum, etc.*)

Ecore

Ecore est une *implémentation* d'EMOF avec

- ▶ une syntaxe concrète graphique (sous forme de diagrammes)
- ▶ une syntaxe concrète sérialisable (en XML)
- ▶ une API en Java, pour manipuler des modèles et des méta-modèles programmatiquement
- ▶ divers outils d'édition de méta-modèles et de modèles
- ▶ une intégration complète dans l'IDE Eclipse

Ecore

Ecore est une *implémentation* d'EMOF avec

- ▶ une syntaxe concrète graphique (sous forme de diagrammes)
- ▶ une syntaxe concrète sérialisable (en XML)
- ▶ une API en Java, pour manipuler des modèles et des méta-modèles programmatiquement
- ▶ divers outils d'édition de méta-modèles et de modèles
- ▶ une intégration complète dans l'IDE Eclipse

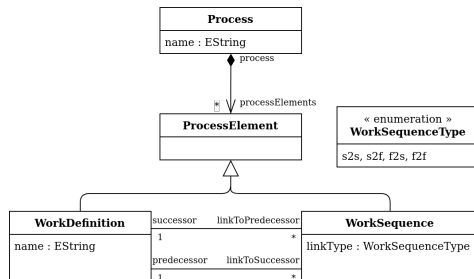
De nombreux outils se basent sur Ecore !

⇒ *nous en verrons quelques uns au fil de l'UE*

Diagrammes Ecore

Diagrammes type ER/UML

- ▶ Concepts (classes) = boîtes
- ▶ Relations = flèches + arité
- ▶ Propriétés = attributs
- ▶ Autres types : énumération, types primitifs (EString, EInt, etc.)



EMOF – Classes, Propriétés

Une *classe* représente un *concept*

- ▶ une classe a un *nom* et des *propriétés*
- ▶ une classe peut être *abstraite*

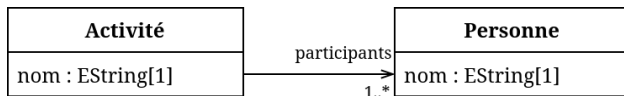
Une *propriété* est un aspect de ce concept

- ▶ une propriété a un *nom* et un *type* (primitif ou datatype)
- ▶ une propriété a une *arité*

Activité
nom : EString[1]
créneau : EDate[*]
fun : EInt

Note : *ça ressemble à UML, mais il y a de subtiles différences...*

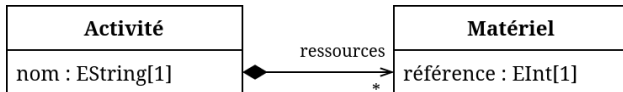
EMOF – Relations/Associations



Une *relation d'association* relie deux classes et représente un lien de collaboration entre deux concepts

- ▶ une association est associée à un *nom* et à une *arité*
- ▶ une association est *directionnelle*

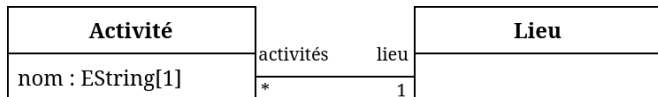
EMOF – Relations/Compositions



Une *relation de composition* est un type particulier d'association, et représente une forme de *contenance*

- ▶ une composition a les mêmes caractéristiques qu'une association (nom, arité, directionnalité)
- ▶ dans un méta-modèle correctement formé, **toute classe est contenue directement ou indirectement dans un élément** (appelé élément *racine*)

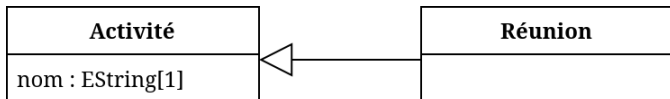
EMOF – Relations/Dualité



Deux associations entre deux classes peuvent être *duales* (ou *opposit*) : elles sont reliées de sorte que si on connaît l'une, on connaît l'autre

- ▶ les compositions aussi peuvent présenter une relation duale
- ▶ on fait des relations duales dès qu'on peut, car c'est très pratique !

EMOF – Relations/Héritage



Une classe peut *hériter* d'une autre

- ▶ une classe fille peut hériter de plusieurs classes mères
- ▶ la classe fille contient toutes les propriétés de les classes mères (en plus des siennes)
- ▶ la classe fille est en relation avec toutes les classes en relation avec les classes mères

EMOF – Datatypes

Un *datatype* représente un concept « primitif », basique et général

- ▶ les *énumérations* sont un type particulier de datatype
- ▶ un datatype peut apparaître comme type d'une *propriété* (pas de relation entre un datatype et une classe)
- ▶ EMOF (et Ecore) propose divers datatype nativement : entiers, flottants, chaînes de caractères, booléens...

« enumeration »
CréneauType
heure, jour, semaine

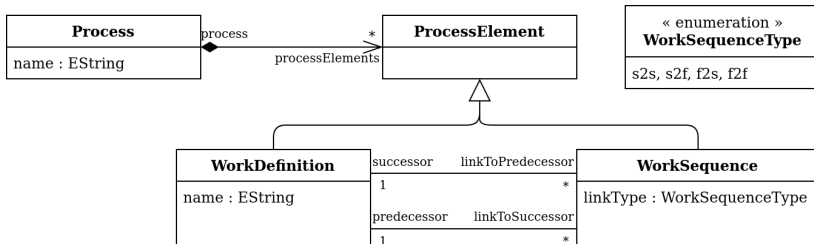
« datatype »
Date

Sommaire

- ① Introduction
- ② Principes Fondateurs
- ③ EMOF et Ecore
- ④ Sémantiques des Modèles
 - Un Exemple : Process Description Language
 - Sémantique Statique
 - Sémantique Opérationnelle
 - Limitations
- ⑤ Les Réseaux de Pétri
- ⑥ Transformations de Modèles

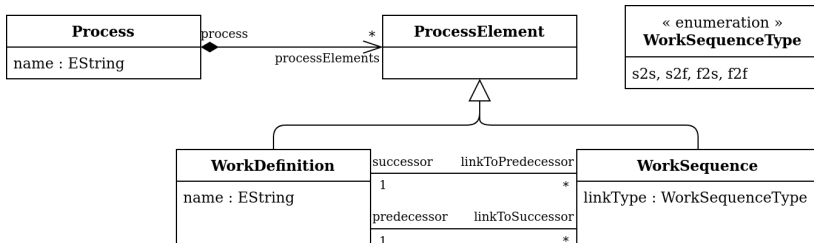
(Simple)PDL, le Modèle de Tâches

Modèle de tâche = « *Process Description Language* » (simple)



(Simple)PDL, le Modèle de Tâches

Modèle de tâche = « *Process Description Language* » (simple)



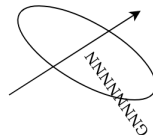
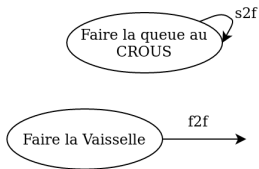
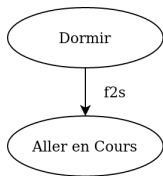
Note : il manque la gestion des ressources !

SimplePDL – Questions

- 1) Le modèle a-t-il un sens ?

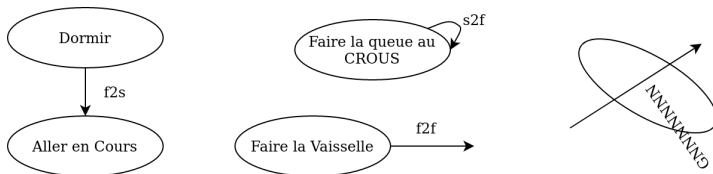
SimplePDL – Questions

1) Le modèle a-t-il un sens ?



SimplePDL – Questions

1) Le modèle a-t-il un sens ?

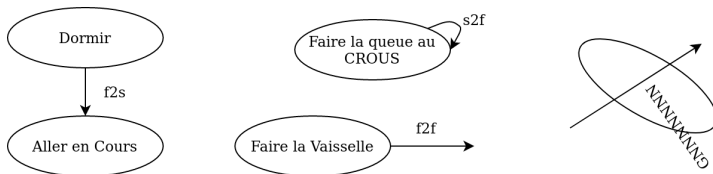


Un méta-modèle établit des **contraintes**

$\Rightarrow$ une *WS* a forcément **1** successeur et **1** prédécesseur

SimplePDL – Questions

1) Le modèle a-t-il un sens ?



Un méta-modèle établit des **contraintes**

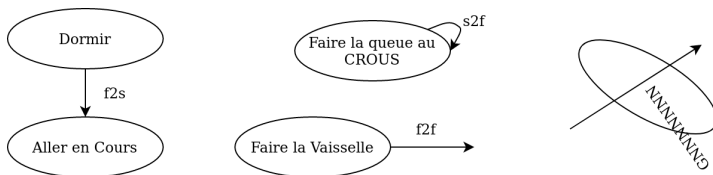
⇒ *une WS a forcément 1 successeur et 1 prédécesseur*

On doit parfois en rajouter...

⇒ *comment éviter les flèches réflexives ?*

SimplePDL – Questions

1) Le modèle a-t-il un sens ?



Un méta-modèle établit des **contraintes**

⇒ *une WS a forcément 1 successeur et 1 prédécesseur*

On doit parfois en rajouter...

⇒ *comment éviter les flèches réflexives ?*

L'ensemble des contraintes définit la **sémantique statique**

Sémantique statique avec OCL

OCL (Object Constraint Language) est un standard de définition de contrainte défini par l'OMG.

Sémantique statique avec OCL

OCL (Object Constraint Language) est un standard de définition de contrainte défini par l'OMG.

```
context WorkDefinition
inv WDNameValid:
  self.name <> null and
  self.name.matches('[a-zA-Z_][a-zA-Z0-9_]*')

context WorkSequence
inv WSNonRef1:
  self.predecessor <> self.successor

context Process
inv UniqueName:
  self.processElements
  ->select(p | p.ocIsKind(WorkDefinition))
  ->collect(p | p.ocAsType(WorkDefinition))
  ->forall(n1, n2 |
    n1.name = n2.name implies n1 = n2)
```

Sémantique statique avec OCL

OCL (Object Constraint Language) est un standard de définition de contrainte défini par l'OMG.

```
context WorkDefinition
inv WDNameValid:
  self.name <> null and
  self.name.matches('[a-zA-Z_][a-zA-Z0-9_]*')

context WorkSequence
inv WSNonRef1:
  self.predecessor <> self.successor

context Process
inv UniqueName:
  self.processElements
  ->select(p | p.ocIsKind(WorkDefinition))
  ->collect(p | p.ocAsType(WorkDefinition))
  ->forall(n1, n2 |
    n1.name = n2.name implies n1 = n2)
```

- Permet d'écrire des contraintes (*inv*) pour un type d'objet (*context*)

Sémantique statique avec OCL

OCL (Object Constraint Language) est un standard de définition de contrainte défini par l'OMG.

```
context WorkDefinition
inv WDNameValid:
  self.name <> null and
  self.name.matches('[a-zA-Z_][a-zA-Z0-9_]*')

context WorkSequence
inv WSNonRefl:
  self.predecessor <> self.successor

context Process
inv UniqueName:
  self.processElements
  ->select(p | p.ocIsKind(WorkDefinition))
  ->collect(p | p.ocAsType(WorkDefinition))
  ->forall(n1, n2 |
    n1.name = n2.name implies n1 = n2)
```

- ▶ Permet d'écrire des contraintes (*inv*) pour un type d'objet (*context*)
- ▶ Pensé pour EMOF, complètement intégré

Sémantique statique avec OCL

OCL (Object Constraint Language) est un standard de définition de contrainte défini par l'OMG.

```
context WorkDefinition
inv WDNameValid:
  self.name <> null and
  self.name.matches('[a-zA-Z_][a-zA-Z0-9_]*')

context WorkSequence
inv WSNonRef1:
  self.predecessor <> self.successor

context Process
inv UniqueName:
  self.processElements
  ->select(p | p.ocIsKind(WorkDefinition))
  ->collect(p | p.ocAsType(WorkDefinition))
  ->forall(n1, n2 |
    n1.name = n2.name implies n1 = n2)
```

- ▶ Permet d'écrire des contraintes (*inv*) pour un type d'objet (*context*)
- ▶ Pensé pour EMOF, complètement intégré
- ▶ Paradigme déclaratif/fonctionnel

Sémantique statique avec OCL

OCL (Object Constraint Language) est un standard de définition de contrainte défini par l'OMG.

```
context WorkDefinition
inv WDNameValid:
  self.name <> null and
  self.name.matches('[a-zA-Z_][a-zA-Z0-9_]*')

context WorkSequence
inv WSNonRefl:
  self.predecessor <> self.successor

context Process
inv UniqueName:
  self.processElements
  ->select(p | p.ocIsKind(WorkDefinition))
  ->collect(p | p.ocAsType(WorkDefinition))
  ->forall(n1, n2 |
    n1.name = n2.name implies n1 = n2)
```

- ▶ Permet d'écrire des contraintes (*inv*) pour un type d'objet (*context*)
- ▶ Pensé pour EMOF, complètement intégré
- ▶ Paradigme déclaratif/fonctionnel

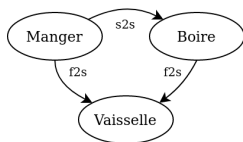
Implémentation : OCLinEcore, CompleteOCL, ...

SimplePDL – Questions

2) Un scénario est-il correct vis-à-vis du modèle ?

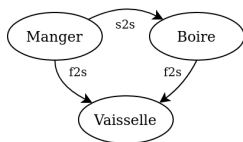
SimplePDL – Questions

2) Un scénario est-il correct vis-à-vis du modèle ?



SimplePDL – Questions

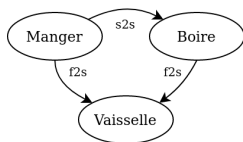
2) Un scénario est-il correct vis-à-vis du modèle ?



Nécessite de définir ce qu'on entend par « correct », + comment exprimer un scénario...

SimplePDL – Questions

2) Un scénario est-il correct vis-à-vis du modèle ?

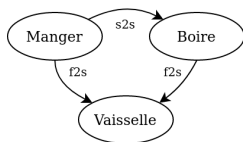


Nécessite de définir ce qu'on entend par « correct », + comment exprimer un scénario...

(oui avec des modèles bien sûr avec des modèles)

SimplePDL – Questions

2) Un scénario est-il correct vis-à-vis du modèle ?



Nécessite de définir ce qu'on entend par « correct », + comment exprimer un scénario...

(oui avec des modèles bien sûr avec des modèles)

On cherche à dire *ce que représente* “formellement” les instances du méta-modèles

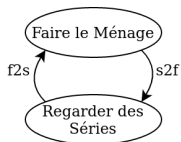
⇒ on parle parfois de sémantique **dynamique** ou **opérationnelle**, ou d'**ontologie**

SimplePDL – Questions

3) Existe-t-il des scénarios qui ne peuvent pas terminer ?

SimplePDL – Questions

3) Existe-t-il des scénarios qui ne peuvent pas terminer ?

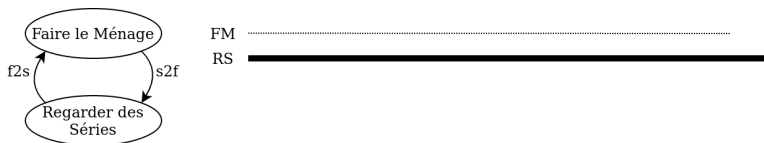


FM

RS **—————**

SimplePDL – Questions

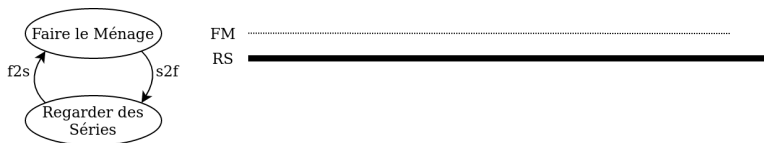
3) Existe-t-il des scénarios qui ne peuvent pas terminer ?



Voilà une question compliquée ! Peut-on contraindre un modèle (= raffiner notre méta-modèle) pour rejeter les processus infaisables ?

SimplePDL – Questions

3) Existe-t-il des scénarios qui ne peuvent pas terminer ?



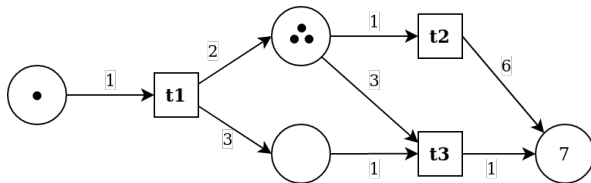
Voilà une question compliquée ! Peut-on contraindre un modèle (= raffiner notre méta-modèle) pour rejeter les processus infaisables ?

Notre méta-modèle *n'est pas adapté* pour répondre à cette question :
 $\Rightarrow$ y a-t-il d'autres méta-modèles plus adaptés ?

Sommaire

- ① Introduction
- ② Principes Fondateurs
- ③ EMOF et Ecore
- ④ Sémantiques des Modèles
- ⑤ Les Réseaux de Pétri
 - Définitions de Base
 - Propriétés
 - Utilisations des Réseaux de Pétri
- ⑥ Transformations de Modèles

Définitions



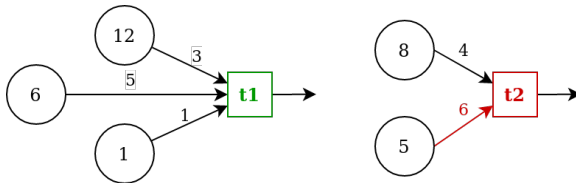
Définition – Réseau P/T

Un réseau P/T se compose de **places** $\bigcirc$, de **transitions** $\square$ et de **flèches pondérées** $\xrightarrow{n}$ reliant les deux, en alternance.

Définition – Jetons, Marquage

Chaque place présente un certain nombre de **jetons** $\bullet$ (≥ 0).
Une configuration du réseau P/T s'appelle un **marquage**.

Sémantique Opérationnelle – Franchissabilité

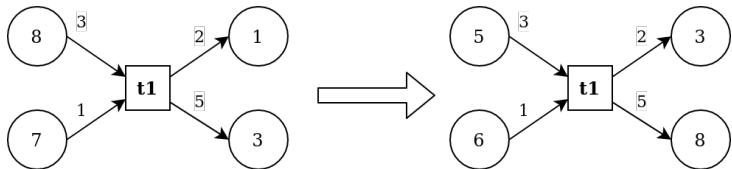


Définition – Franchissabilité

Une transition est **franchissable** lorsque, pour toute flèche entrante i de poids n_i , la place origine de la flèche contient un nombre de jeton *supérieur ou égal* à n_i .

$\Rightarrow$ $t1$ est franchissable, $t2$ ne l'est pas

Sémantique Opérationnelle – Exécution



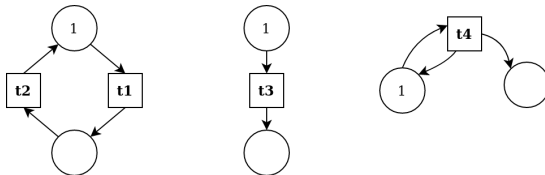
Définition – Franchissement

Lorsque l'on **franchit** une transition, on

1. **retire** de chaque place antécédente le poids de la flèche associée
2. **ajoute** à chaque place suivante le poids de la flèche associée

$\Rightarrow$ *franchir une transition transforme le marquage !*

Propriétés des Réseaux de Pétri



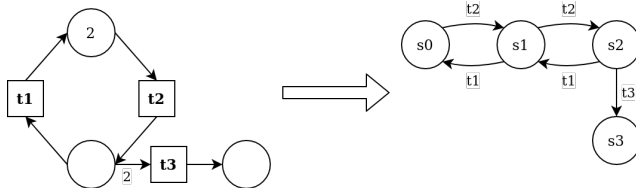
Définition – Vivacité

Un réseau P/T est **vivace** si il n'existe pas d'état (= de marquage) duquel on ne peut plus progresser (= franchir des transitions).

Définition – Bornage

Un réseau P/T est **borné** si aucune place ne peut recevoir un nombre arbitraire de jetons.

Graphe de Marquage



Définition – Graphe de Marquage

Le **graphe de marquage** (ou de **couverture**) d'un réseau P/T a

- un noeud s pour chaque marquage possible du réseau
- une flèche $s \xrightarrow{t} s'$ si franchir la transition t dans le marquage s donne le marquage s'

Graphe de Marquage – Propriétés

Définition – Atteignabilité

Un marquage s' est **atteignable** depuis s si, et seulement si il existe une séquence de marquages et de transitions $(t_1, s_1, t_2, s_2, \dots, s_{n-1}, t_n)$ tel que

$$s \xrightarrow{t_1} s_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} s'$$

Graphe de Marquage – Propriétés

Définition – Atteignabilité

Un marquage s' est **atteignable** depuis s si, et seulement si il existe une séquence de marquages et de transitions $(t_1, s_1, t_2, s_2, \dots, s_{n-1}, t_n)$ tel que

$$s \xrightarrow{t_1} s_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} s'$$

Propriété de Vivacité

Un réseau de P/T est vivace si, et seulement si son graphe de marquage n'a pas de **noeud piège atteignable**.

Propriété de Bornage

Un réseau de P/T est borné si, et seulement si son graphe de marquage est **fini**.

Pourquoi faire des réseaux de Pétri

Les réseaux de Pétri sont très bons pour modéliser tout ce qui consiste en une *succession d'états*

⇒ *programmes, agents communicants, processus...*

Pourquoi faire des réseaux de Pétri

Les réseaux de Pétri sont très bons pour modéliser tout ce qui consiste en une *succession d'états*

⇒ *programmes, agents communicants, processus...*

Il existe des outils performants pour répondre à des questions sur les réseaux de Pétri

Nous utiliserons Tina <https://projects.laas.fr/tina>

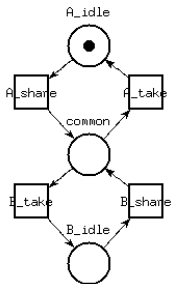
Pourquoi faire des réseaux de Pétri

Les réseaux de Pétri sont très bons pour modéliser tout ce qui consiste en une *succession d'états*

⇒ *programmes, agents communicants, processus...*

Il existe des outils performants pour répondre à des questions sur les réseaux de Pétri

Nous utiliserons Tina <https://projects.laas.fr/tina>



```
# net Test2, 3 places, 4 transitions, 8 arcs
# bounded, live, reversible
# abstraction      count      props      psets      dead      live #
# states           3          3          3          0          3 #
# transitions       4          4          4          0          4 #
```

```
lritputer % ~/projects/tina/test:
10026 ◯ selt test2.ktz -f "[ ] <> common"
Selt version 3.7.0 -- 05/19/22 -- LAAS/CNRS
ktz loaded, 3 states, 4 transitions
0.001s
TRUE
0.000s
lritputer % ~/projects/tina/test:
10027 ◯ selt test2.ktz -f "- <> A_idle"
Selt version 3.7.0 -- 05/19/22 -- LAAS/CNRS
ktz loaded, 3 states, 4 transitions
0.001s
FALSE
state 0: A_idle
-A_share ... (preserving A_idle)->
state 1: common
[accepting all]
```

Un Peu de Recul

On a défini les réseaux de Pétri (en langage naturel + un peu de maths), c'est un **méta-modèle**.

⇒ On a vu plusieurs exemples (= modèles) de réseaux de Pétri.

Un Peu de Recul

On a défini les réseaux de Pétri (en langage naturel + un peu de maths), c'est un **méta-modèle**.

⇒ *On a vu plusieurs exemples (= modèles) de réseaux de Pétri.*

Les réseaux de Pétri permettent de représenter tout processus à état de manière assez abstraite !

⇒ *Les jetons peuvent représenter une ressource, un tour, une permission...*

Un Peu de Recul

On a défini les réseaux de Pétri (en langage naturel + un peu de maths), c'est un **méta-modèle**.

⇒ *On a vu plusieurs exemples (= modèles) de réseaux de Pétri.*

Les réseaux de Pétri permettent de représenter tout processus à état de manière assez abstraite !

⇒ *Les jetons peuvent représenter une ressource, un tour, une permission...*

Les réseaux de Pétri permettent de répondre à des questions spécifiques : vivacité/terminaison, atteignabilité, etc.

⇒ *il existe des outils très puissants pour les manipuler et pour répondre à ces questions !*

Rapport à SimplePDL

- ▶ SimplePDL est « confortable » à manipuler ; les réseaux P/T sont un peu compliqués et « bruts »
 $\Rightarrow$ *pratiquement pas de sémantique ontologique !*

Rapport à SimplePDL

- ▶ SimplePDL est « confortable » à manipuler ; les réseaux P/T sont un peu compliqués et « bruts »
⇒ *pratiquement pas de sémantique ontologique !*
- ▶ SimplePDL permet de répondre à des questions simples ; les réseaux P/T à des questions plus subtiles...
⇒ *typiquement, détecter des réseaux qui ne progressent plus...*

Rapport à SimplePDL

- ▶ SimplePDL est « confortable » à manipuler ; les réseaux P/T sont un peu compliqués et « bruts »
⇒ *pratiquement pas de sémantique ontologique !*
- ▶ SimplePDL permet de répondre à des questions simples ; les réseaux P/T à des questions plus subtiles...
⇒ *typiquement, détecter des réseaux qui ne progressent plus...*

Idée : on pourrait trouver un moyen de « convertir » un modèle de processus en réseau de Pétri

⇒ *si la transformation est « correcte », alors les propriétés du réseaux impliquent des propriétés du modèle de processus !*

Sommaire

- ① Introduction
- ② Principes Fondateurs
- ③ EMOF et Ecore
- ④ Sémantiques des Modèles
- ⑤ Les Réseaux de Pétri
- ⑥ Transformations de Modèles
 - Principes de Base
 - Exemple
- ⑦ Application : Mini-Projet

Principes de Base

Un modèle représente des données sous une certaine forme
 $\Rightarrow$ *on peut vouloir changer de représentation*

Principes de Base

Un modèle représente des données sous une certaine forme

$\Rightarrow$ *on peut vouloir changer de représentation*

Par ex. : on représente des points 3D en coordonnées cartésiennes

- ▶ on peut les projeter en 2D ou en 4D
- ▶ on peut les représenter en coordonnées sphériques, cylindriques
- ▶ on peut les convertir en commande pour un outil de dessin (SVG)
- ▶ etc.

Principes de Base

Un modèle représente des données sous une certaine forme

⇒ *on peut vouloir changer de représentation*

Par ex. : on représente des points 3D en coordonnées cartésiennes

- ▶ on peut les projeter en 2D ou en 4D
- ▶ on peut les représenter en coordonnées sphériques, cylindriques
- ▶ on peut les convertir en commande pour un outil de dessin (SVG)
- ▶ etc.

Idée : en IDM, les *données* sont abstraites par des *méta-modèles* ; les *procédés* sur ces données sont abstraits par des ***transformations***

Définition (informelle)

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Définition (informelle)

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

*⇒ en général les modèles en sortie ont un lien avec ceux en entrée
(sinon ça n'a pas grand intérêt en fait)*

Définition (informelle)

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

⇒ en général les modèles en sortie ont un lien avec ceux en entrée (sinon ça n'a pas grand intérêt en fait)

Une transformation pose des contraintes sur les modèles en E/S (typiquement, des MM auxquels les modèles doivent se conformer)
⇒ une transformation accepte tout modèle conforme, et produit des modèles conformes en retour

Remarques

Une transformation...

Remarques

Une transformation...

- ▶ est un lien entre des méta-modèles

Remarques

Une transformation...

- ▶ est un lien entre des méta-modèles
- ▶ peut prendre n'importe quelle forme

Remarques

Une transformation...

- ▶ est un lien entre des méta-modèles
- ▶ peut prendre n'importe quelle forme
- ▶ peut aussi induire des propriétés additionnelles
⇒ *par ex. équivalences sémantiques, connexité, intégrité, etc.*

Remarques

Une transformation...

- ▶ est un lien entre des méta-modèles
- ▶ peut prendre n'importe quelle forme
- ▶ peut aussi induire des propriétés additionnelles
⇒ *par ex. équivalences sémantiques, connexité, intégrité, etc.*
- ▶ peut « oublier » des informations de l'entrée

Remarques

Une transformation...

- ▶ est un lien entre des méta-modèles
- ▶ peut prendre n'importe quelle forme
- ▶ peut aussi induire des propriétés additionnelles
⇒ *par ex. équivalences sémantiques, connexité, intégrité, etc.*
- ▶ peut « oublier » des informations de l'entrée
- ▶ peut associer un ou plusieurs objets en entrée à un ou plusieurs objets en sortie

Autres Remarques

En général, on exprime une transformation au niveau des méta-modèles, sous forme de règles *Entrées* $\longrightarrow$ *Sorties*

Autres Remarques

En général, on exprime une transformation au niveau des méta-modèles, sous forme de règles $Entrées \longrightarrow Sorties$

- ▶ $WorkDefinition(nom) \longrightarrow Place(a,1), Place(b), Transition(t1,a,b), \dots$
- ▶ $WorkSequence(src,dest) \longrightarrow Arc(src,dest)$
- ▶ ...

$\Rightarrow$ *il existe des outils pour écrire des transformations ! (ex. ATL)*

Autres Remarques

En général, on exprime une transformation au niveau des méta-modèles, sous forme de règles *Entrées* $\longrightarrow$ *Sorties*

- ▶ $WorkDefinition(nom) \longrightarrow Place(a,1), Place(b), Transition(t1,a,b), \dots$
- ▶ $WorkSequence(src,dest) \longrightarrow Arc(src,dest)$
- ▶ ...

$\Rightarrow$ *il existe des outils pour écrire des transformations ! (ex. ATL)*

On peut vouloir transformer des modèles en **textes** (on parle alors de *model-to-text*²).

$\Rightarrow$ *conceptuellement, le MM est la grammaire si le texte est structuré, ou alors juste l'encodage...*

2. sinon, on parle de *model-to-model*

Retour au Problème Initial

Idée :

Retour au Problème Initial

Idée :

- ▶ on a un MM de SimplePDL

Retour au Problème Initial

Idée :

- ▶ on a un MM de SimplePDL
- ▶ on a(ura) un MM pour les réseaux de Pétri

Retour au Problème Initial

Idée :

- ▶ on a un MM de SimplePDL
 - ▶ on a(ura) un MM pour les réseaux de Pétri
- ⇒ on écrit une transformation de l'un à l'autre !

Retour au Problème Initial

Idée :

- ▶ on a un MM de SimplePDL
- ▶ on a(ura) un MM pour les réseaux de Pétri
- ⇒ on écrit une transformation de l'un à l'autre !

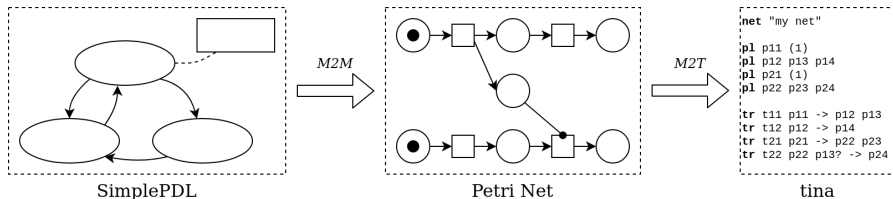
⇒ *en plus de ça, on peut transformer des modèles de réseau dans un format (textuel) utilisable par un outil*

Retour au Problème Initial

Idée :

- ▶ on a un MM de SimplePDL
- ▶ on a(ura) un MM pour les réseaux de Pétri
- ⇒ on écrit une transformation de l'un à l'autre !

⇒ *en plus de ça, on peut transformer des modèles de réseau dans un format (textuel) utilisable par un outil*

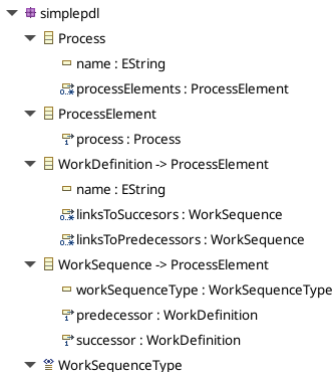


Sommaire

- ① Introduction
- ② Principes Fondateurs
- ③ EMOF et Ecore
- ④ Sémantiques des Modèles
- ⑤ Les Réseaux de Pétri
- ⑥ Transformations de Modèles
- ⑦ Application : Mini-Projet
 - Les technologies utilisées
 - Présentation du sujet

Modèles et Sémantique Statique

- Framework EMF +
outil (Complete) OCL
- Exprimer un méta-modèle (classes,
attributs, références, arités...) +
sémantique statique (contraintes)



```

import 'SimplePDL.ecore'

package simplePDL

context Process
inv nom_correct:
self.name <> null and
self.name.matches('[A-Za-z_][A-Za-z0-9_]*')

context WorkSequence
inv non_reflexif:
self.predecessor <> self.successor

endpackage
  
```

Syntaxes Concrètes

Un méta-modèle est une syntaxe *abstraite*.

$\Rightarrow$ *comment manipule-t-on des modèles concrètement ?*

Syntaxes Concrètes

Un méta-modèle est une syntaxe *abstraite*.

⇒ *comment manipule-t-on des modèles concrètement ?*

Syntaxes Concrètes

Une **syntaxe concrète** est *une* représentation des instances d'un méta-modèle, qui permet de les manipuler.

Une syntaxe concrète est associée à un méta-modèle.

Syntaxes Concrètes

Un méta-modèle est une syntaxe *abstraite*.

⇒ *comment manipule-t-on des modèles concrètement ?*

Syntaxes Concrètes

Une **syntaxe concrète** est *une* représentation des instances d'un méta-modèle, qui permet de les manipuler.

Une syntaxe concrète est associée à un méta-modèle.

Note importante : on parle de *syntaxe*, mais il n'y a pas de restriction sur cette représentation (graphique, textuelle, formats structurés, ...)

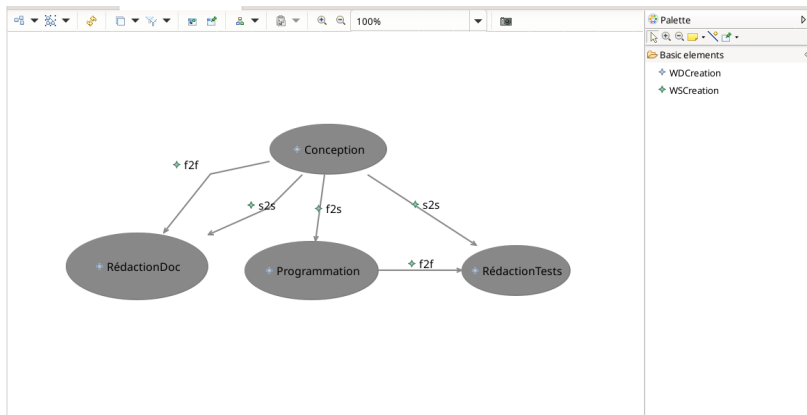
Éditeur Réflexif

- ▶ Éditeur *no-code*, généré à la volée par Eclipse
- ▶ Permet d'éditer simplement des modèles
- ▶ Compatible avec OCL
- ▶ Pratique mais pas hyper ergonomique...

platform:/resource/fr.n7.gls.simplepdl/Process.xml	
- Process Développement - Work Definition Conception - Work Definition RédactionDoc - Work Definition Programmation - Work Definition RédactionTests - Work Sequence s2s - Work Sequence f2f - Work Sequence s2s - Work Sequence f2s - Work Sequence f2f	
Property	Value
Links To Predecessors	Work Sequence f2s
Links To Successors	Work Sequence f2f
Name	Programmation
Process	Process Développement

Syntaxe Concrète Graphique

- ▶ Outil Sirius (<https://www.eclipse.org/sirius/>)
- ▶ Éditeur low-code, définit avec un modèle (ODesign)
- ▶ Affichage configurable, calques, palette...



Syntaxe Concrète Textuelle

- ▶ Outil Xtext (<https://www.eclipse.org/Xtext/>)
- ▶ Point d'entrée : une grammaire
- ▶ Résultat : MM + outil (coloration syntaxique) qui crée des modèles

```
process Developpement {  
  wd Conception  
  wd RedactionDoc  
    starts when Conception starts  
    finishes when Conception finishes  
  wd Developpement  
    starts when Conception finishes  
  wd RedactionDoc  
    starts when Conception starts  
    finishes when Developpement finishes  
}
```

platform:/resource/fr.n7.samples/developpement.pdl

- ▼ ♦ Process Developpement
 - ♦ Work Definition Conception
 - ▼ ♦ Work Definition RedactionDoc
 - ♦ Work Sequence starts
 - ♦ Work Sequence finishes
 - ▼ ♦ Work Definition Developpement
 - ♦ Work Sequence starts
 - ▼ ♦ Work Definition RedactionDoc
 - ♦ Work Sequence starts
 - ♦ Work Sequence finishes

Représentation Java

- ▶ MM > Architecture Java
- ▶ Permet de manipuler des modèles dans des programmes !
- ▶ Les modèles peuvent être faits avec d'autres syntaxes !

```
SimplepdlFactory myFactory = SimplepdlFactory.eINSTANCE;

Process process = myFactory.createProcess();
process.setName("Test");
resource.getContents().add(process);

WorkDefinition wd1 = myFactory.createWorkDefinition();
wd1.setName("A1");

WorkDefinition wd2 = myFactory.createWorkDefinition();
wd2.setName("A2");

process.getProcessElements().add(wd1);
process.getProcessElements().add(wd2);

WorkSequence seq = myFactory.createWorkSequence();
seq.setPredecessor(wd1);
seq.setSuccessor(wd2);
seq.setWorkSequenceType(WorkSequenceType.F2F);

process.getProcessElements().add(seq);
```

Transformation Modèle-à-Modèle

- ▶ Outil ATL (<https://www.eclipse.org/atl/>)
- ▶ Langage déclaratif sous forme de règles
 - ⇒ on décrit la *logique* plutôt que le *moyen*
- ▶ ATL se charge de la gestion des ressources, ordonnancement des règles, etc.

```
-- @path petrinet=/fr.n7.gls.petrinet/PetriNet.ecore
-- @path simplepdl=/fr.n7.gls.simplepdl/SimplePDL.ecore

module simplepdl2petrinet;
create OUT : petrinet from IN : simplepdl;

rule Process2Net {
  from proc : simplepdl!Process
  to   net : petrinet!PetriNet(name <- proc.name)
}

rule WD2Net {
  from wd : simplepdl!WorkDefinition
  to
    p_ready : petrinet!Place(net <- wd.process, name <- wd.name + '_ready', marking <- 1),
    p_started : petrinet!Place(net <- wd.process, name <- wd.name + '_started'),
    p_finished : petrinet!Place(net <- wd.process, name <- wd.name + '_finished'),
    t_start : petrinet!Transition(net <- wd.process, name <- wd.name + '_start'),
    t_finish : petrinet!Transition(net <- wd.process, name <- wd.name + '_finish'),
    a_ready_start : petrinet!Arc(net <- wd.process, source <- p_ready , target <- t_start),
    a_start_started : petrinet!Arc(net <- wd.process, source <- t_start , target <- p_started),
    a_started_finish : petrinet!Arc(net <- wd.process, source <- p_started, target <- t_finish),
    a_finish_finished : petrinet!Arc(net <- wd.process, source <- t_finish , target <- p_finished)
}
```

Transformation Modèle-à-Texte

- ▶ Outil Acceleo (<https://www.eclipse.org/acceleo/>)
- ▶ Langage de *template* (“texte à trous”)

```
[module generate('http://simplepd/')]

```

```
[template public generateElement(aProcess : Process)]
[comment @main/]
[file (aProcess.name + '.dot', false, 'UTF-8')]
digraph [aProcess.name /] {
  [for (wd : WorkDefinition | aProcess.getWorkDefinitions())]
  [wd.toDot() /]
  [/for]
  [for (ws : WorkSequence | aProcess.getWorkSequences())]
  "[ws.predecessor.name/] -> "[ws.successor.name/] ["'/'label="["ws.workSequenceType/"]['']"/];
  [/for]
}
[/file]
[/template]

[query public getWorkDefinitions(p : Process) : Collection(WorkDefinition) =
p.processElements->select(e | e.oclIsKindOf(WorkDefinition))->collect(e | e.oclAsType(WorkDefinition)) /]

[query public getWorkSequences(p : Process) : Collection(WorkSequence) =
p.processElements->select(e | e.oclIsKindOf(WorkSequence))->collect(e | e.oclAsType(WorkSequence)) /]

[template private toDot(w : WorkDefinition)]
"[w.name /]"f["'/'shape=ellipse["'']"/];
[/template]
```

```
digraph Développement {
  "Conception"[shape=ellipse];
  "RédactionDoc"[shape=ellipse];
  "Programmation"[shape=ellipse];
  "RédactionTests"[shape=ellipse];
  "Conception" -> "RédactionDoc" [label="s2s"];
  "Conception" -> "RédactionTests" [label="f2f"];
  "Conception" -> "RédactionTests" [label="s2s"];
  "Conception" -> "Programmation" [label="f2s"];
  "Programmation" -> "RédactionTests" [label="f2f"];
}
```

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)
2. méta-modéliser les réseaux de Pétri (Ecore) + contraintes (OCL)

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)
2. méta-modéliser les réseaux de Pétri (Ecore) + contraintes (OCL)
3. définir des syntaxes concrètes pour SimplePDL : graphique (Sirius), textuelle (Xtext)

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)
2. méta-modéliser les réseaux de Pétri (Ecore) + contraintes (OCL)
3. définir des syntaxes concrètes pour SimplePDL : graphique (Sirius), textuelle (Xtext)
4. définir une transformation M2M SimplePDL $\longrightarrow$ Réseau Pétri (ATL et Java)

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)
2. méta-modéliser les réseaux de Pétri (Ecore) + contraintes (OCL)
3. définir des syntaxes concrètes pour SimplePDL : graphique (Sirius), textuelle (Xtext)
4. définir une transformation M2M SimplePDL $\longrightarrow$ Réseau Pétri (ATL et Java)
5. définir une transformation M2T Réseau Pétri $\longrightarrow$ tina (Acceleo)

Présentation

BUT : *une chaîne de vérification de modèles de processus*

Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)
2. méta-modéliser les réseaux de Pétri (Ecore) + contraintes (OCL)
3. définir des syntaxes concrètes pour SimplePDL : graphique (Sirius), textuelle (Xtext)
4. définir une transformation M2M SimplePDL $\longrightarrow$ Réseau Pétri (ATL et Java)
5. définir une transformation M2T Réseau Pétri $\longrightarrow$ tina (Acceleo)
6. générer des propriétés à vérifier (Acceleo)

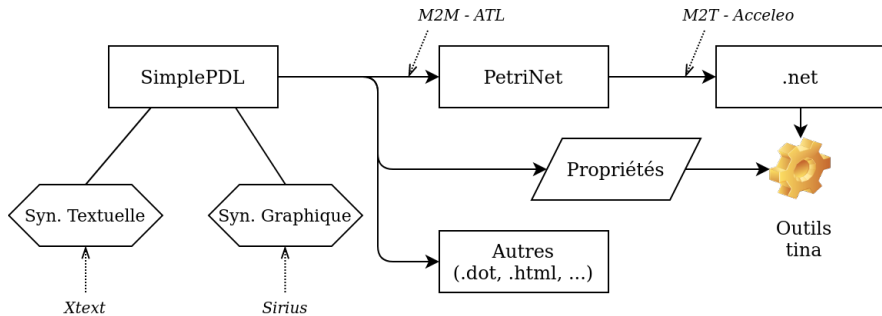
Présentation

BUT : *une chaîne de vérification de modèles de processus*

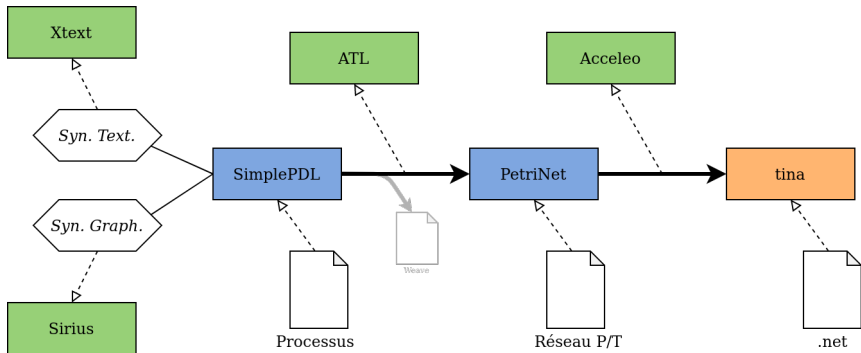
Grandes étapes :

1. méta-modéliser SimplePDL *avec les ressources* (Ecore) + contraintes (OCL)
2. méta-modéliser les réseaux de Pétri (Ecore) + contraintes (OCL)
3. définir des syntaxes concrètes pour SimplePDL : graphique (Sirius), textuelle (Xtext)
4. définir une transformation M2M SimplePDL $\longrightarrow$ Réseau Pétri (ATL et Java)
5. définir une transformation M2T Réseau Pétri $\longrightarrow$ tina (Acceleo)
6. générer des propriétés à vérifier (Acceleo)
7. d'autres fonctionnalités si vous avez envie!
 $\Rightarrow$ *générer du DOT, du HTML, des listes, des rapports, ...*

Un Schéma pour Résumer



Un Autre Schéma (j'aime bien les schémas)



Sommaire

- 1 Introduction
- 2 Principes Fondateurs
- 3 EMOF et Ecore
- 4 Sémantiques des Modèles
- 5 Les Réseaux de Pétri
- 6 Transformations de Modèles
- 7 Application : Mini-Projet
- 8 Conclusion

Conclusion

L'IDM est un domaine à part entière de l'informatique, actif en industrie et en recherche.

Conclusion

L'IDM est un domaine à part entière de l'informatique, actif en industrie et en recherche.

L'idée est de rapprocher l'utilisateur de la technique (trop souvent décorrelés), d'intégrer le domaine dans le produit, d'automatiser les traitements.

Conclusion

L'IDM est un domaine à part entière de l'informatique, actif en industrie et en recherche.

L'idée est de rapprocher l'utilisateur de la technique (trop souvent décorrelés), d'intégrer le domaine dans le produit, d'automatiser les traitements.

Domaines d'application très variés, à toutes les étapes du cycle de développement

⇒ *ex. très concret : spécifier des réseaux virtuels et générer automatiquement la configuration nécessaire*